

Pontifícia Universidade Católica do Rio Grande do Sul

Escola Politécnica

Laboratório de Redes de Computadores

Interceptação e Defesa de um Protocolo de Aplicação

Relatório da Fase 1 — Reconhecimento e Escuta Passiva

Integrantes:

Rosana Schreiner Budant

Luísa Kirsch Silva Zarth

Leonardo Nunes Pasa

Porto Alegre — outubro de 2026

1. Objetivo

Este relatório documenta a Fase 1 do trabalho, cujo propósito é demonstrar, na prática, por que um protocolo de aplicação sem proteção é vulnerável. Para isso, o grupo implementou um par cliente-servidor que se comunica em texto claro e uma ferramenta de interceptação passiva (sniffer) construída com socket raw, capaz de ler o tráfego e extrair a informação sensível que nele circula. O resultado evidencia a quebra de confidencialidade.

2. Ambiente de testes (topologia)

O enunciado prevê um ambiente isolado com, no mínimo, três máquinas: cliente, servidor e observador. Conforme combinado, o grupo montou esse ambiente dentro de um único GitHub Codespace, utilizando três terminais no lugar de três máquinas virtuais. Os três processos executam no mesmo container Linux e se comunicam pela interface de loopback (lo, 127.0.0.1), que é onde o sniffer realiza a captura.

- Terminal 1 — Servidor (papel da VM-servidor): escuta em 0.0.0.0:9999.
- Terminal 2 — Sniffer (papel da VM-observador): raw socket que lê o tráfego da interface lo. Executa com privilégio de root (sudo).
- Terminal 3 — Cliente (papel da VM-cliente): conecta em 127.0.0.1:9999 e dispara os comandos do protocolo.

Por se tratar de loopback, cada mensagem é observada duas vezes na captura (no envio e na recepção pela própria interface lo). Isso é um comportamento esperado do loopback, e não duplicação de dados gerada pela aplicação.

3. O protocolo de aplicação

O protocolo escolhido é um repositório chave-valor com autenticação simples, combinação de duas das sugestões do enunciado. É um protocolo texto claro sobre TCP, na porta 9999, em que cada mensagem é uma linha terminada por quebra de linha. Ele carrega deliberadamente informação sensível (credenciais no LOGIN e valores arbitrários no SET), para que a interceptação tenha conteúdo relevante a demonstrar.

Comandos: LOGIN usuario senha, SET chave valor, GET chave, DEL chave e QUIT. Com exceção do LOGIN, todos exigem uma sessão autenticada. As respostas iniciadas por “OK” indicam sucesso; as iniciadas por “Erro!” indicam falha.

4. Interceptação passiva

4.1. Sniffer próprio (raw socket)

O sniffer abre um socket raw (AF_PACKET / SOCK_RAW), lê os quadros da rede, confirma que são IPv4 e TCP, localiza o início do payload a partir dos cabeçalhos IP e TCP e, filtrando pela porta 9999, imprime o conteúdo da aplicação. Rodando-o junto do servidor e do cliente, toda a sessão aparece legível. Abaixo, um trecho representativo da saída do sniffer:

```
IP ORIGEM | IP DESTINO | TEMPO | DADOS CONTIDOS
```

```
127.0.0.1 | 127.0.0.1 | 12:19:33 | LOGIN alice senha123
IP ORIGEM | IP DESTINO | TEMPO | DADOS CONTIDOS
127.0.0.1 | 127.0.0.1 | 12:19:34 | SET cartao 1234-5678-9012-3456
```

A credencial (usuário e senha) e o número de cartão enviado via SET aparecem em texto puro, sem qualquer cifragem.

4.2. Evidência com Wireshark

Para confirmar a leitura pacote a pacote, o tráfego também foi capturado com tcpdump (gerando o arquivo evidencia.pcap) e aberto no Wireshark. A função Follow TCP Stream reconstrói a conversa completa entre cliente e servidor, exibindo-a integralmente em texto claro:

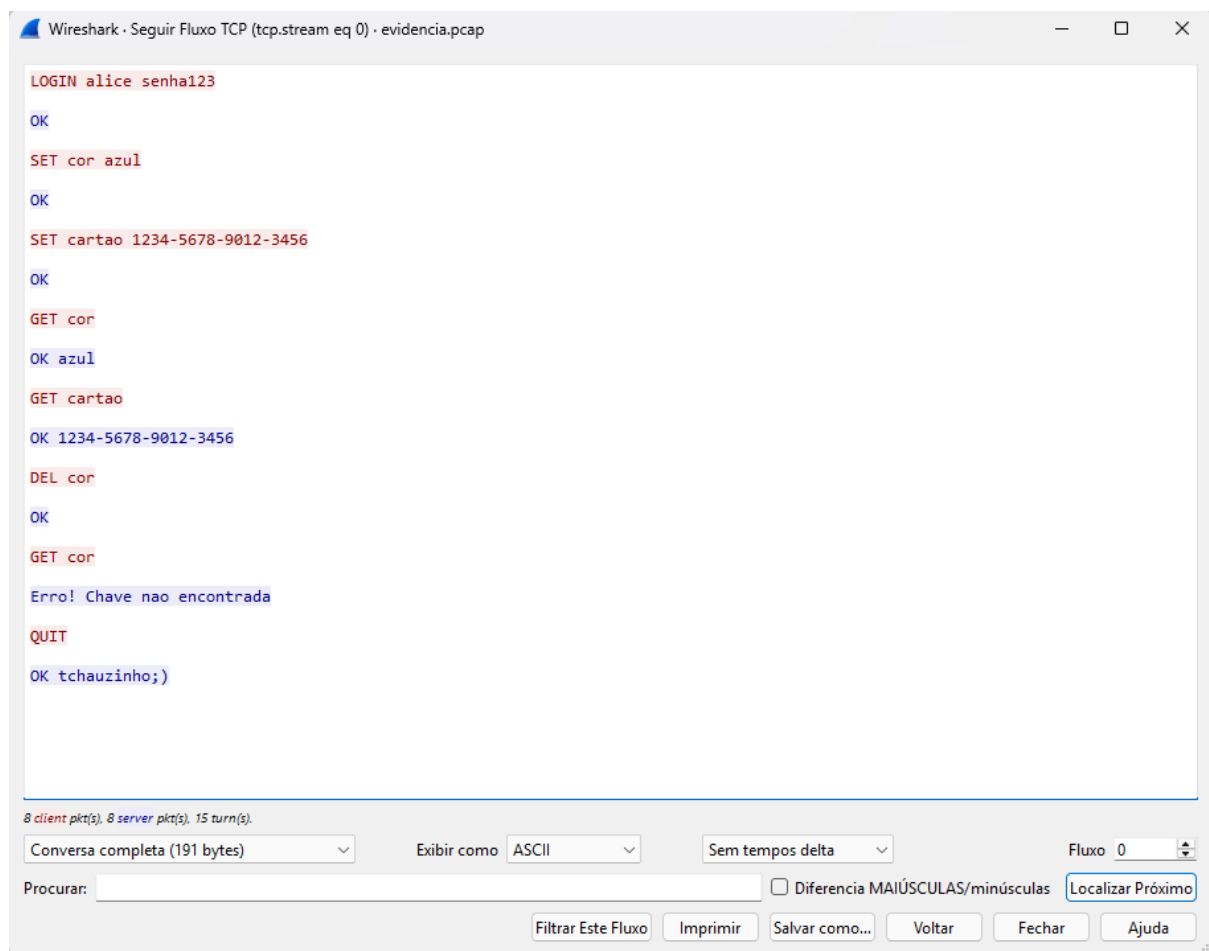


Figura 1 — Follow TCP Stream no Wireshark: a sessão inteira (LOGIN, SET, GET, DEL, QUIT) em texto claro.

A lista de pacotes, filtrada por `tcp.port == 9999`, permite identificar exatamente em qual pacote cada dado sensível trafega. Selecionando o pacote nº 4, o painel de bytes mostra o conteúdo “LOGIN alice senha123” legível:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	74	60280 → 9999 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=2916021854 TSecr=0 WS=128
2	0.000014	127.0.0.1	127.0.0.1	TCP	74	9999 → 60280 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=2916021854 TSecr=2916021854 WS=128
3	0.000027	127.0.0.1	127.0.0.1	TCP	66	60280 → 9999 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=2916021854 TSecr=2916021854
4	0.000159	127.0.0.1	127.0.0.1	TCP	87	60280 → 9999 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=21 TSval=2916021854 TSecr=2916021854
5	0.000180	127.0.0.1	127.0.0.1	TCP	66	9999 → 60280 [ACK] Seq=1 Ack=22 Win=65536 Len=0 TSval=2916021854 TSecr=2916021854
6	0.000226	127.0.0.1	127.0.0.1	TCP	69	9999 → 60280 [PSH, ACK] Seq=1 Ack=22 Win=65536 Len=3 TSval=2916021854 TSecr=2916021854
7	0.000238	127.0.0.1	127.0.0.1	TCP	66	60280 → 9999 [ACK] Seq=22 Ack=4 Win=65536 Len=0 TSval=2916021854 TSecr=2916021854
8	0.000510	127.0.0.1	127.0.0.1	TCP	79	60280 → 9999 [PSH, ACK] Seq=22 Ack=4 Win=65536 Len=13 TSval=2916022354 TSecr=2916021854
9	0.000630	127.0.0.1	127.0.0.1	TCP	69	9999 → 60280 [PSH, ACK] Seq=4 Ack=35 Win=65536 Len=3 TSval=2916022354 TSecr=2916022354
10	0.000648	127.0.0.1	127.0.0.1	TCP	66	60280 → 9999 [ACK] Seq=35 Ack=7 Win=65536 Len=0 TSval=2916022354 TSecr=2916022354
11	0.000856	127.0.0.1	127.0.0.1	TCP	97	60280 → 9999 [PSH, ACK] Seq=35 Ack=7 Win=65536 Len=31 TSval=2916022854 TSecr=2916022354
12	0.000976	127.0.0.1	127.0.0.1	TCP	69	9999 → 60280 [PSH, ACK] Seq=7 Ack=66 Win=65536 Len=3 TSval=2916022855 TSecr=2916022854
13	0.001013	127.0.0.1	127.0.0.1	TCP	66	60280 → 9999 [ACK] Seq=66 Ack=10 Win=65536 Len=0 TSval=2916022855 TSecr=2916022855
14	0.001262	127.0.0.1	127.0.0.1	TCP	74	60280 → 9999 [PSH, ACK] Seq=66 Ack=10 Win=65536 Len=8 TSval=2916023355 TSecr=2916022855
15	0.001379	127.0.0.1	127.0.0.1	TCP	74	9999 → 60280 [PSH, ACK] Seq=10 Ack=74 Win=65536 Len=8 TSval=2916023355 TSecr=2916023355

> Frame 4: Packet, 87 bytes on wire (696 bits), 87 bytes captured (696 bits)

Internet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)

Ethernet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

Transmission Control Protocol, Src Port: 60280, Dst Port: 9999, Seq: 1, Ack: 1, Len: 21

Data (21 bytes)

0000 00 00 00 00 00 00 00 00 00 00 00 00 08 00 45 00E-

0010 00 49 88 14 40 00 00 04 06 b4 98 7f 00 00 01 7f 00I-x-

0020 00 01 be 78 27 0f db 97 06 31 49 8c 1a ac 00 18-I-

0030 02 00 fe 3d 00 00 01 01 08 0a ad ce f6 5e ad ce-LOGIN alice se

0040 f6 5e ac 4f 47 49 4e 20 61 6c 69 63 65 20 73 65nha123-

0050 6e 68 61 31 32 33 0a

Figura 2 — Lista de pacotes; o pacote nº 4 (Data, 21 bytes) carrega a credencial em texto claro.

Pacotes de referência: o pacote nº 4 transporta a credencial (LOGIN alice senha123) e o pacote nº 11 transporta o dado sensível do SET (cartao 1234-5678-9012-3456). Ambos são legíveis diretamente no payload, sem necessidade de qualquer chave ou decodificação.

5. Análise

Tanto o sniffer próprio quanto o Wireshark leem o conteúdo integral da comunicação. Isso caracteriza a quebra de confidencialidade: qualquer observador com acesso ao meio por onde o tráfego passa consegue recuperar credenciais e dados sensíveis sem esforço, pois nada é cifrado. Como o protocolo também não possui autenticação forte nem verificação de integridade, ele fica igualmente exposto aos ataques das fases seguintes.

A Fase 3 tratará exatamente dessas fraquezas, protegendo o canal com TLS sobre o socket TCP e substituindo o armazenamento de senhas em texto claro por senhas com hash, de modo que a mesma captura passe a exibir apenas dados ilegíveis.

6. Conclusão

A Fase 1 atingiu seu objetivo: o par cliente-servidor e o sniffer em raw socket foram implementados e, em conjunto com a evidência do Wireshark, demonstram de forma inequívoca que o protocolo trafega informação sensível em texto claro. O ambiente de três terminais no Codespace reproduziu os papéis de cliente, servidor e observador exigidos pelo enunciado. Fica evidenciada a quebra de confidencialidade, que fundamenta a necessidade das defesas a serem implementadas na Fase 3.